Reflection

TL;DR
Mit Hilfe der Reflection-API kann man Programme zur **Laufzeit** inspizieren und Eigenschaften von Elementen wie Klassen oder Methoden abfragen, aber auch Klassen instantiieren und Methoden aufrufen, die eigentlich auf `private` gesetzt sind oder die beispielsweise mit einer bestimmten Annotation markiert sind.
Die Laufzeitumgebung erzeugt zu jedem Typ ein Objekt der Klasse `java.lang.Class`. Über dieses `Class`-Objekt einer Klasse können dann Informationen über diese Klasse abgerufen werden, beispielsweise welche Konstruktoren, Methoden und Attribute es gibt.
Man kann über auch Klassen zur Laufzeit nachladen, die zur Compile-Zeit nicht bekannt waren. Dies bietet sich beispielsweise für User-definierte Plugins an.
Reflection ist ein mächtiges Werkzeug. Durch das Arbeiten mit Strings und die Interaktion/Inspektion zur *Laufzeit* verliert man aber viele Prüfungen, die der Compiler normalerweise zur Compile-Zeit vornimmt. Auch das Refactoring wird dadurch eher schwierig.

Videos

- VL Reflection
- Demo Reflection
- Demo Class-Loader

Unterlagen

- PDF-Version

Ausgaben und Einblicke zur Laufzeit

```
public class FactoryBeispielTest {
    @Test
    public void testGetTicket() {
        fail("not implemented");
    }
}
```

```
@Target(value = ElementType.METHOD)
@Retention(value = RetentionPolicy.RUNTIME)
public @interface Wuppie {}
```

Reflection wird allgemein genutzt, um zur Laufzeit von Programmen Informationen über Klassen/Methoden/… zu bestimmen. Man könnte damit auch das Verhalten der laufenden Programme ändern oder Typen instantiieren und/oder Methoden aufrufen …

Wenn Sie nicht (mehr) wissen, wie man eigene Annotationen definiert, schauen Sie doch bitte einfach kurz im Handout zu Annotationen nach :-)

Wer bin ich? ... Informationen über ein Programm (zur Laufzeit)

`java.lang.Class`: Metadaten über Klassen

```
// usual way of life
Studi heiner = new Studi();
heiner.hello();

// let's use reflection
try {
    Object eve = Studi.class.getDeclaredConstructor().newInstance();
    Method m = Studi.class.getDeclaredMethod("hello");
    m.invoke(eve);
} catch (ReflectiveOperationException ignored) {}
```

Für jeden Typ instantiiert die JVM eine nicht veränderbare Instanz der Klasse `java.lang.Class`, über die Informationen zu dem Typ abgefragt werden können.

Dies umfasst u.a.:

- Klassenname
- Implementierte Interfaces
- Methoden
- Attribute
- Annotationen
- ...

`java.lang.Class` bildet damit den Einstiegspunkt in die Reflection.

Vorgehen

1. Gewünschte Klasse über ein `Class`-Objekt laden
2. Informationen abrufen (welche Methoden, welche Annotationen, ...)
3. Eine Instanz dieser Klasse erzeugen, und
4. Methoden aufrufen

Das Vorgehen umfasst vier Schritte: Zunächst die gewünschte Klasse über ein `Class`-Objekt laden und anschließend Informationen abrufen (etwa welche Methoden vorhanden sind, welche Annotationen annotiert wurden, ...) und bei Bedarf eine Instanz dieser Klasse erzeugen sowie Methoden aufrufen.

Ein zweiter wichtiger Anwendungsfall (neben dem Abfragen von Informationen und Aufrufen von Methoden) ist das Laden von Klassen, die zur Compile-Zeit nicht mit dem eigentlichen Programm verbunden sind. Auf diesem Weg kann beispielsweise ein Bildbearbeitungsprogramm zur Laufzeit dynamisch Filter aus einem externen Ordner laden und nutzen, oder der Lexer kann die Tokendefinitionen zur Laufzeit einlesen (d.h. er könnte mit unterschiedlichen Tokensätzen arbeiten, die zur Compile-Zeit noch gar nicht definiert sind). Damit werden die Programme dynamischer.

Schritt 1: *Class*-Objekt erzeugen und Klasse laden

```
// Variante 1 (package.MyClass dynamisch zur Laufzeit laden)
Class<?> c = Class.forName("package.MyClass");


// Variante 2 (Objekt)
MyClass obj = new MyClass();
Class<?> c = obj.getClass();

// Variante 3 (Klasse)
Class<?> c = MyClass.class;
```

=> Einstiegspunkt der Reflection API

Eigentlich wird nur in **Variante 1 die über den String angegebene Klasse dynamisch von der Laufzeitumgebung (nach-) geladen** (muss also im gestarteten Programm nicht vorhanden sein). Die angegebene Klasse muss aber in Form von Byte-Code an der angegebenen Stelle (Ordner `package`, Dateiname `MyClass.class`) vorhanden sein.

Die anderen beiden Varianten setzen voraus, dass die jeweilige Klasse **bereits geladen** ist (also ganz normal mit den restlichen Sourcen zu Byte-Code (`.class`-Dateien) kompiliert wurde und mit dem Programm geladen wurde).

Alle drei Varianten ermöglichen die Introspektion der jeweiligen Klassen zur Laufzeit.

Schritt 2: In die Klasse reinschauen

```
// Studi-Klasse dynamisch (nach-) laden
Class<?> c = Class.forName("reflection.Studi");


// Parametersatz für Methode zusammenbasteln
Class<?>[] paramT = new Class<?>[] { String.class };

// public Methode aus dem **Class**-Objekt holen
Method pubMethod = c.getMethod("setName", paramT);
// beliebige Methode aus dem **Class**-Objekt holen
Method privMethod = c.getDeclaredMethod("setName", paramT);


Method[] publicMethods = c.getMethods();  // all public methods (incl.
    inherited)
Method[] allMethods = c.getDeclaredMethods();  // all methods (excl. inherited)
```

- `public` Methode laden (auch von Superklasse/Interface geerbt): `Class<?>.getMethod(String, Class<?>[])`
- Beliebige (auch `private`) Methoden (in der Klasse selbst deklariert): `Class<?>.getDeclaredMethod(...)`

Anmerkung: Mit `Class<?>.getDeclaredMethods()` erhalten Sie alle Methoden, die direkt in der Klasse deklariert werden (ohne geerbte Methoden!), unabhängig von deren Sichtbarkeit. Mit `Class<?>.getMethods`

`()` erhalten Sie dagegen alle **`public`** Methoden, die in der Klasse selbst oder ihren Superklassen bzw. den implementierten Interfaces deklariert sind.

Vgl. Javadoc `getMethods` und `getDeclaredMethods`.

Die Methoden-Arrays können Sie nach bestimmten Eigenschaften durchsuchen, bzw. auf das Vorhandensein einer bestimmten Annotation prüfen (etwa mit `isAnnotationPresent()`) etc.

Analog können Sie weitere Eigenschaften einer Klasse abfragen, beispielsweise Attribute (`Class<?>.getDeclaredFields()`) oder Konstruktoren (`Class<?>.getDeclaredConstructors()`).

Schritt 3: Instanz der geladenen Klasse erzeugen

```
// Class-Objekt erzeugen
Class<?> c = Class.forName("reflection.Studi");


// Variante 1
Studi s = (Studi) c.newInstance();

// Variante 2
Constructor<?> ctor = c.getConstructor();
Studi s = (Studi) ctor.newInstance();

// Variante 3
Class<?>[] paramT = new Class<?>[] {String.class, int.class};
Constructor<?> ctor = c.getDeclaredConstructor(paramT);
Studi s = (Studi) ctor.newInstance("Beate", 42);
```

Parameterlose, öffentliche Konstruktoren:

- `Class<?>.newInstance()` (seit Java9 *deprecated*!)
- `Class<?>.getConstructor()` => `Constructor<?>.newInstance()`

Sonstige Konstruktoren:

Passenden Konstruktor explizit holen: `Class<?>.getDeclaredConstructor(Class<?>[])`, Parametersatz zusammenbasteln (hier nicht dargestellt) und aufrufen `Constructor<?>.newInstance(...)`

Unterschied *new* und *Constructor.newInstance()*:

`new` ist nicht identisch zu `Constructor.newInstance()`: **`new`** kann Dinge wie Typ-Prüfung oder Auto-Boxing mit erledigen, während man dies bei `Constructor.newInstance()` selbst explizit angeben oder erledigen muss.

Vgl. docs.oracle.com/javase/tutorial/reflect/member/ctorTrouble.html.

Schritt 4: Methoden aufrufen ...

```
// Studi-Klasse dynamisch (nach-) laden
Class<?> c = Class.forName("reflection.Studi");
// Studi-Objekt anlegen (Defaultkonstruktor)
Studi s = (Studi) c.newInstance();
// Parametersatz für Methode zusammenbasteln
Class<?>[] paramT = new Class<?>[] { String.class };
// Methode aus dem **Class**-Objekt holen
Method method = c.getMethod("setName", paramT);


// Methode auf dem **Studi**-Objekt aufrufen
method.invoke(s, "Holgi");
```

Die Reflection-API bietet neben dem reinen Zugriff auf (alle) Methoden noch viele weitere Möglichkeiten. Beispielsweise können Sie bei einer Methode nach der Anzahl der Parameter und deren Typ und Annotationen fragen etc. ... Schauen Sie am besten einmal selbst in die API hinein.

Demo: reflection.ReflectionDemo (https://github.com/Programmiermethoden-CampusMinden/Prog2-Lecture/blob/master/lecture/java-classic/src/reflection/ReflectionDemo.java)

Hinweis: Klassen außerhalb des Classpath laden

```
File folder = new File("irgendwo");
URL[] ua = new URL[]{folder.toURI().toURL()};

URLClassLoader ucl = URLClassLoader.newInstance(ua);
Class<?> c1 = Class.forName("org.wuppie.Fluppie", true, ucl);
Class<?> c2 = ucl.loadClass("org.wuppie.Fluppie");
```

Bemerkung zu Ordnerstruktur und Classpath; Demo: reflection.ClassLoaderDemo (https://github.com/Programmiermethoden-CampusMinden/Prog2-Lecture/blob/master/lecture/java-classic/src/reflection/ClassLoaderDemo.java)

Mit `Class.forName("reflection.Studi")` können Sie die Klasse `Studi` im Package `reflection` laden. Dabei muss sich aber die entsprechende `.class`-Datei (samt der der Package-Struktur entsprechenden Ordnerstruktur darüber) **im Java-Classpath** befinden!

Mit einem weiteren `ClassLoader` können Sie auch aus Ordnern, die sich nicht im Classpath befinden, `.class`-Dateien laden. Dies geht dann entweder wie vorher über `Class.forName()`, wobei hier der neue Class-Loader als Parameter mitgegeben wird, oder direkt über den neuen Class-Loader mit dessen Methode `loadClass()`.

Licht und Schatten

Nützlich:

- Erweiterbarkeit: Laden von “externen” (zur Kompilierzeit unbekannter) Klassen in eine Anwendung

- Klassen-Browser, Debugger und Test-Tools

Nachteile:

- Verlust von Kapselung, Compiler-Unterstützung und Refactoring
- Performance: Dynamisches Laden von Klassen etc.
- Sicherheitsprobleme/-restriktionen

Reflection ist ein nützliches Werkzeug. Aber: **Gibt es eine Lösung ohne Reflection, wähle diese!**

Wrap-Up

- Inspektion von Programmen zur Laufzeit: **Reflection**
  - `java.lang.Class`: Metadaten über Klassen
  - Je Klasse ein `Class`-Objekt
  - Informationen über Konstruktoren, Methoden, Felder
  - Anwendung: Laden und Ausführen von zur Compile-Zeit unbekanntem Code
  - Vorsicht: Verlust von Refactoring und Compiler-Zusicherungen!

Zum Nachlesen

- Oracle Corporation (2024)
- Inden (2013, Kap. 8)

Lernziele

- k2: Probleme beim Einsatz von Reflection
- k2: Bedeutung der verschiedenen Exceptions beim Aufruf von Methoden per Reflection
- k3: Inspection von Klassen zur Laufzeit mit Reflection
- k3: Einbindung von zur Compilezeit unbekannten Klassen, Aufruf von Konstruktoren und Methoden (mit und ohne Parameter/Rückgabewerte)

Challenges

In den Vorgaben finden Sie eine einfache Implementierung für einen Taschenrechner mit Java-Swing. Dieser Taschenrechner kann nur mit `int`-Werten rechnen. Der Taschenrechner verfügt über keinerlei vordefinierte mathematische Operationen (Addieren, Subtrahieren etc.).

Erstellen Sie eigene mathematische Operationen, die `IOperation` implementieren. Jede Ihrer Klassen soll mit einer Annotation vermerkt werden, in welcher der Name der jeweiligen Operation gespeichert wird.

Der Taschenrechner lädt seine Operationen dynamisch über die statische Methode `OperationLoader.loadOperations` ein. In den Vorgaben ist diese Methode noch nicht ausimplementiert. Implementieren Sie die Funktion so, dass sie mit Hilfe von Reflection Ihre Operationen einliest. Geben Sie dazu den Ordner an, in dem die entsprechenden `.class`-Dateien liegen. (Dieser Ordner soll sich außerhalb Ihres Java-Projekts befinden!) Verändern Sie nicht die Signatur der Methode.

Ihre Operation-Klassen dürfen Sie nicht vorher bekannt machen. Diese müssen in einem vom aktuellen Projekt separierten Ordner/Projekt liegen.

Quellen

Inden, M. 2013. *Der Weg zum Java-Profi*. 2. Aufl. dpunkt.verlag.

Oracle Corporation. 2024. „The Java Tutorials“. 2024. https://docs.oracle.com/javase/tutorial/.

Unless otherwise noted, this work is licensed under CC BY-SA 4.0.

> **Last modified:** 2ce9f3f (Tooling: use original folder structure for generated pdf's, 2025-04-17)